SYSTEM DESIGN · TAKE-HOME REVIEW

# A Modern Ecommerce System

Bounded Contexts, Aggregates, Sequence Diagrams & Physical Architecture

Revised submission · adds the sequence-diagram and architecture views requested by the exercise brief

GAP ADDRESSED

**What changed and why.** The exercise deliverable explicitly asks for "bounded contexts and their relationships *via sequence diagrams*", plus notes that "architecture or choreography diagrams are a plus." The original write-up modeled contexts, aggregates and commands well, but represented context relationships only as `flowchart` diagrams (topology), not as time-ordered `sequenceDiagram` views (interaction order, request/response, sync vs. async). This revision keeps every original section and adds: three sequence diagrams (happy path, out-of-stock compensation, ambiguous payment reconciliation) and one physical/component architecture diagram, closing both gaps directly.

## 1 Bounded Contexts and Context Map

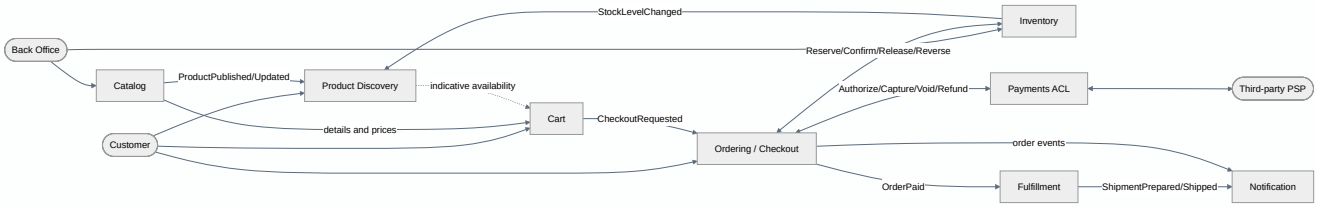
Eight bounded contexts partition the domain along ownership and change-rate boundaries. Each row states what a context is authoritative for and, just as importantly, what it deliberately is *not* responsible for — the negative space is what keeps modules decoupled.

**Table 1 — Bounded contexts and their responsibilities**

| Context | Responsible for | Not responsible for |
|---|---|---|
| **Catalog** | Product master data, descriptions, media, list prices, publishing | Available quantities |
| **Inventory** | Stock levels per product, reservations, restocking, non-overselling invariant | Prices, orders, payments, shipping |
| **Cart** | Customer/guest cart, cart lines, and price snapshots | Guaranteeing availability at checkout |
| **Ordering / Checkout** | Orders, state machine, checkout orchestration, and compensations | Payment details or the physical handling of stock and shipping |
| **Payments** | Abstraction over the payment service, authorization, capture, void/refund, and reconciliation | Stock and availability |
| **Fulfillment** | Preparation and shipping of paid orders, tracking | Payments, pricing, and commercial availability |
| **Product Discovery** | Full-text index on product names and descriptions, filters, indicative availability | Any domain write operation |
| **Notification** | Transactional emails and order/shipping status notifications | Physical order preparation or shipping |

**Figure 1 — Context map (structural view)**

Solid arrows are synchronous calls; dashed arrows are asynchronous domain events. This view answers “who talks to whom”, but not “in what order” — that is what Figures 3–5 add.




## 2 Aggregates, Entities, and Domain Methods

Each context owns exactly one write-model aggregate root that enforces its invariant; no aggregate is shared across a context boundary. This directly satisfies the “Commands” deliverable: every state transition is expressed as a method on an aggregate that emits one or more domain events.

**Table 2 — Aggregates, entities, value objects and invariants**

| Aggregate (root) | Context | Contained entities | Value objects | Key invariant |
|---|---|---|---|---|
| `Product` | Catalog | ProductOption, Media | ProductId, Money, Description, Media | A published product has a name, description, price, and at least one image |
| `StockItem` (one per product) | Inventory | Reservation | ProductId, Quantity, ReservationId, IdempotencyKey | `available = onHand - reserved` and `available ≥ 0` |
| `Cart` | Cart | CartLine | Owner, ProductId, PriceSnapshot | Positive quantities, one line per product, and a cart that has neither expired nor already entered checkout |
| `Order` | Ordering | OrderLine | OrderNumber, Buyer, ShippingAddress, BillingData, OrderTotals, CheckoutId | No payment is requested without an active Reservation for every line; Paid requires confirmed stock and a captured payment; totals cannot change after authorization starts |
| `Payment` | Payments | PaymentAttempt | Money, ProviderReference, IdempotencyKey, PaymentMethodToken | Retries do not cause duplicate charges; active captures cannot exceed the order total |
| `Shipment` | Fulfillment | Package | TrackingReference, ShippingAddress | A shipment is created only for a paid order and cannot be marked as shipped without tracking information |

**Table 3 — Commands (aggregate methods) and emitted events**

| Aggregate | Command / method | Emitted event |
|---|---|---|
| `Product` | `create(name, description, price)` | ProductCreated |
| | `attachMedia(media)` · `updateDetails(...)` | ProductDetailsUpdated |
| | `changePrice(money)` | PriceChanged |
| | `publish()` · `discontinue()` | ProductPublished · ProductDiscontinued |

| | | |
|---|---|---|
| `StockItem` | `receiveStock(qty)` | StockReceived + StockLevelChanged |
| | `reserve(orderId, orderLineId, qty, expiresAt, key)` | StockReserved(reservationId) or StockReservationRejected |
| | `confirmReservation(reservationId, key)` | StockConfirmed + StockLevelChanged |
| | `releaseReservation(reservationId, reason, key)` | StockReleased + StockLevelChanged |
| | `reverseConfirmation(reservationId, reason, key)` | StockConfirmationReversed + StockLevelChanged |
| `Cart` | `addItem(productId, qty, priceSnapshot)` · `changeQuantity(...)` · `removeItem(...)` | CartUpdated |
| | `mergeIntoCustomerCart(customerId)` | CartMerged |
| `Order` | `startCheckout(cart, shipping, billing)` | CheckoutStarted |
| | `confirmAllStockReserved(checkoutId, reservationIds)` | OrderStockReserved |
| | `markAuthorizationPending()` · `markPaymentAuthorized(paymentId)` | PaymentAuthorizationRequested · OrderPaymentAuthorized |
| | `confirmStock()` · `markCapturePending()` | OrderStockConfirmed · PaymentCaptureRequested |
| | `markPaid()` · `rejectOutOfStock(lines)` · `cancel(reason)` | OrderPaid · OrderRejectedOutOfStock · OrderCancelled |
| `Payment` | `authorize(methodToken, amount, key)` | PaymentAuthorized, PaymentDeclined, or PaymentOutcomeUnknown |
| | `capture(key)` · `voidAuthorization(key)` · `refund(amount, key)` | PaymentCaptured/PaymentCaptureFailed · PaymentAuthorizationVoided · PaymentRefunded |
| `Shipment` | `prepare(orderId, packages)` · `markShipped(tracking)` | ShipmentPrepared · OrderShipped |

# 3 Context Interactions — Sequence Diagrams

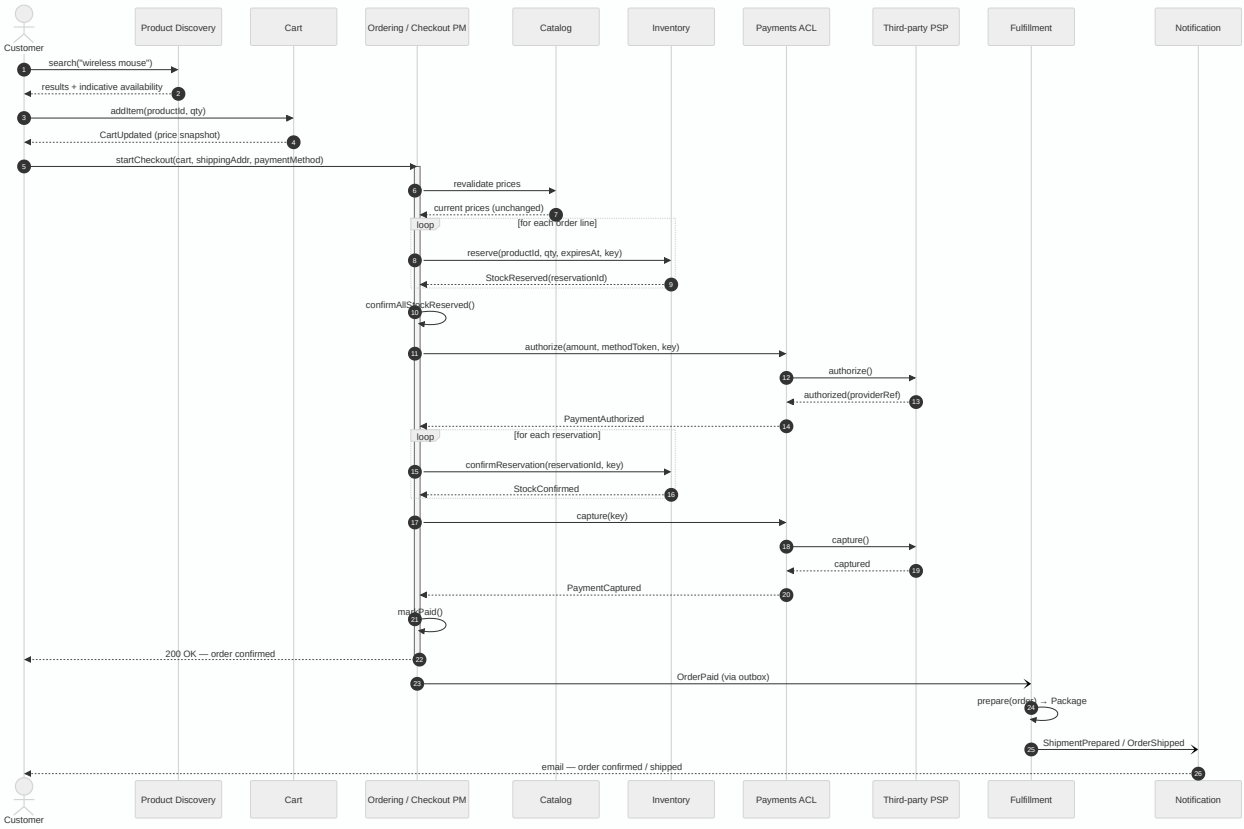
**NEW IN THIS REVISION**

This section is the direct response to the brief's deliverable: “bounded contexts and their relationships (via sequence diagrams).” The context map in Figure 1 shows *topology*; the three diagrams below show *time* — call order, who blocks on whom, and which hops are synchronous vs. fire-and-forget. Read together, they answer the three questions an interviewer will actually ask: what happens on the happy path, what happens when stock runs out, and what happens when the payment service itself is unreliable.

## 3.1 · Happy path: search to shipped notification

**Figure 2 — End-to-end checkout, no failures**

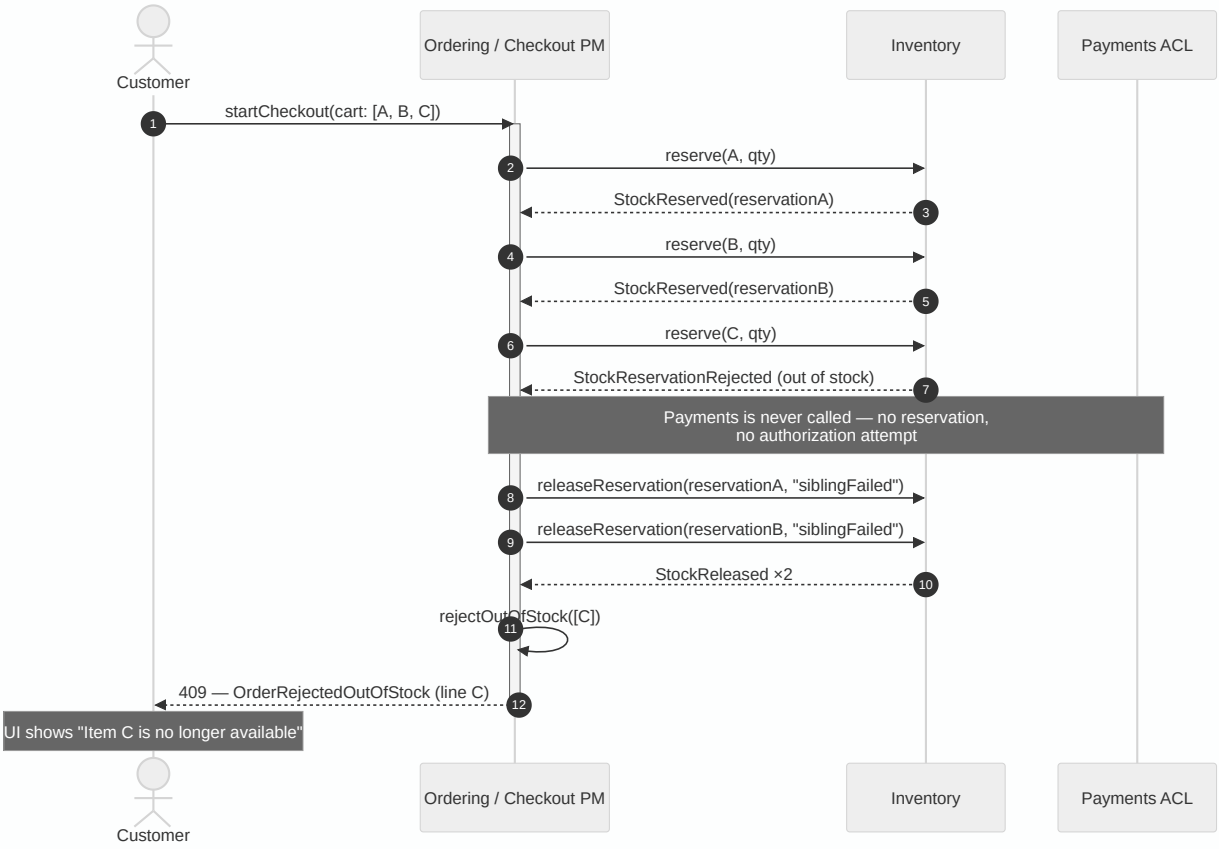
Reservation happens before authorization; authorization happens before capture; capture happens before `OrderPaid`. Fulfillment and Notification are only ever reached through the outbox (dashed async arrows), never synchronously from the checkout path.



## 3.2 · Out-of-stock at checkout (compensation, no payment attempted)

### Figure 3 — Partial reservation failure

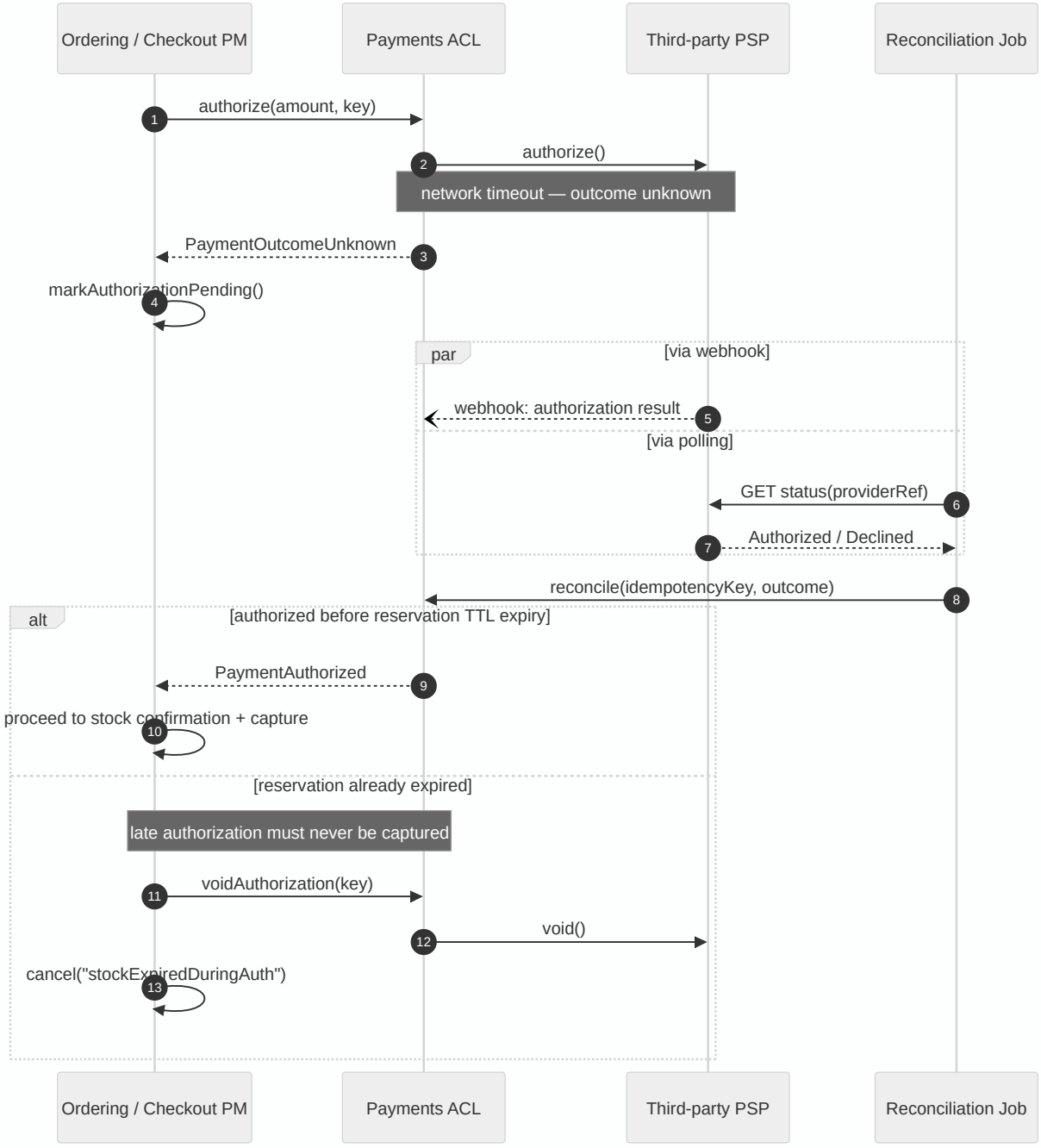
Directly implements the assumption “if the goods are exhausted, prevent payment and indicate that in the UI.” Payments is never called: Ordering releases the reservations it already holds before responding, so there is no partially-valid multi-product reservation and no dangling stock hold.



## 3.3 · Ambiguous payment outcome (reconciliation)

### Figure 4 — PSP timeout, resolved asynchronously

A timeout is not a decline. Stock stays reserved (TTL is the safety net) while a webhook and a polling job race to resolve the real outcome via `ProviderReference` / `IdempotencyKey`. This is the scenario that most tests whether "handling stock is done outside of payments" actually holds under failure.

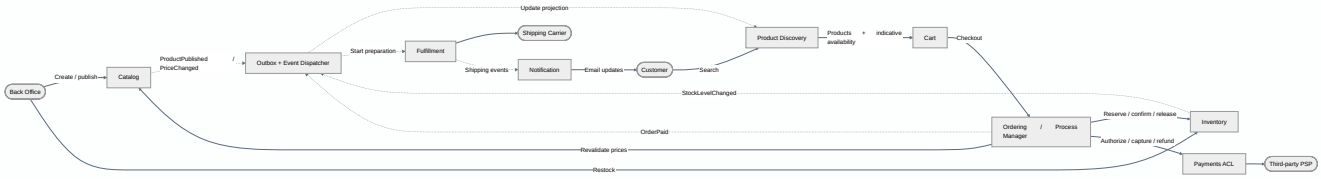


## 4 Event Choreography — Customer Journey

Figures 2–4 show *orchestration*: Ordering drives Inventory and Payments through explicit request/response calls because checkout correctness requires a single decision-maker. Everything downstream of `OrderPaid`, and everything that keeps the search index warm, is instead *choreographed* — contexts react to events they subscribe to, with no central coordinator. Both styles coexist deliberately: orchestration where a strict invariant spans contexts, choreography where contexts are simply eventually consistent.

**Figure 5 — Full customer journey with event choreography**

Dotted arrows are asynchronous domain events flowing through the outbox + event dispatcher; solid arrows are synchronous calls inside the orchestrated checkout path.




### Compensations and edge cases

- **Partial reservation across multiple products.** If checkout has obtained reservationA and reservationB, but the hold for product C fails, the process manager releases the two individual reservations already created. There is no partially valid multi-product reservation. The TTL remains a safety net if the process stops before compensation.
- **Orphan reservation.** Every `Reservation` has an `expiresAt`; a scheduler moves active reservations to `Expired` and makes the quantity available again. Correctness does not depend on delivery of the release command.
- **Ambiguous payment service outcome.** A timeout is not equivalent to a decline. Stock is not released immediately: webhooks and status queries reconcile the attempt using `ProviderReference` and `IdempotencyKey`. If the reservation expires before the outcome is known, a late authorization is voided and never captured.
- **Failure while confirming multiple products.** The authorization is voided, reservations that have not yet been confirmed are released, while those already confirmed are compensated through `reverseConfirmation`. These operations are idempotent and recorded in the stock movement audit log.
- **Definitive capture failure.** The authorization is voided, and stock confirmations are compensated before the order is cancelled. If the capture outcome is unknown, no compensation is performed: the order remains `CapturePending` until an external event or job establishes whether the charge occurred. A capture that succeeded but cannot be used is handled through an explicit refund.
- **Price change.** Prices are revalidated between add-to-cart and checkout; if they differ, the customer must reconfirm. Order totals become immutable before authorization.

## 5 Physical Architecture and Ownership

> **NEW IN THIS REVISION**
>
> The brief lists an architecture diagram as a plus. Figure 6 makes the “modular monolith” decision concrete: one deployable process, module-owned schemas, and a durable outbox/inbox instead of a volatile in-memory bus — so the same code can later publish to an external broker without a rewrite.

**Recommendation: a modular monolith, not microservices.** Each bounded context is a module with its own public API, ownership, and logical schema; aggregates are not shared. Checkout uses a persistent process manager. Asynchronous effects travel through outbox/inbox and versioned event contracts.

In the first release, a durable dispatcher inside the application reads the outbox and invokes handlers in other modules. It is not a volatile in-memory bus, because messages and progress are persisted. When operational isolation or independent scaling becomes necessary, the same transport adapter can publish to an external broker. Protobuf, partitioning, and consumer groups then become infrastructure choices — not prerequisites for the monolith.

**Figure 6 — Component & deployment view**

One deployable process, four team-owned modules, one schema per bounded context, and a single outbox/dispatcher/inbox spine carrying every cross-module event.

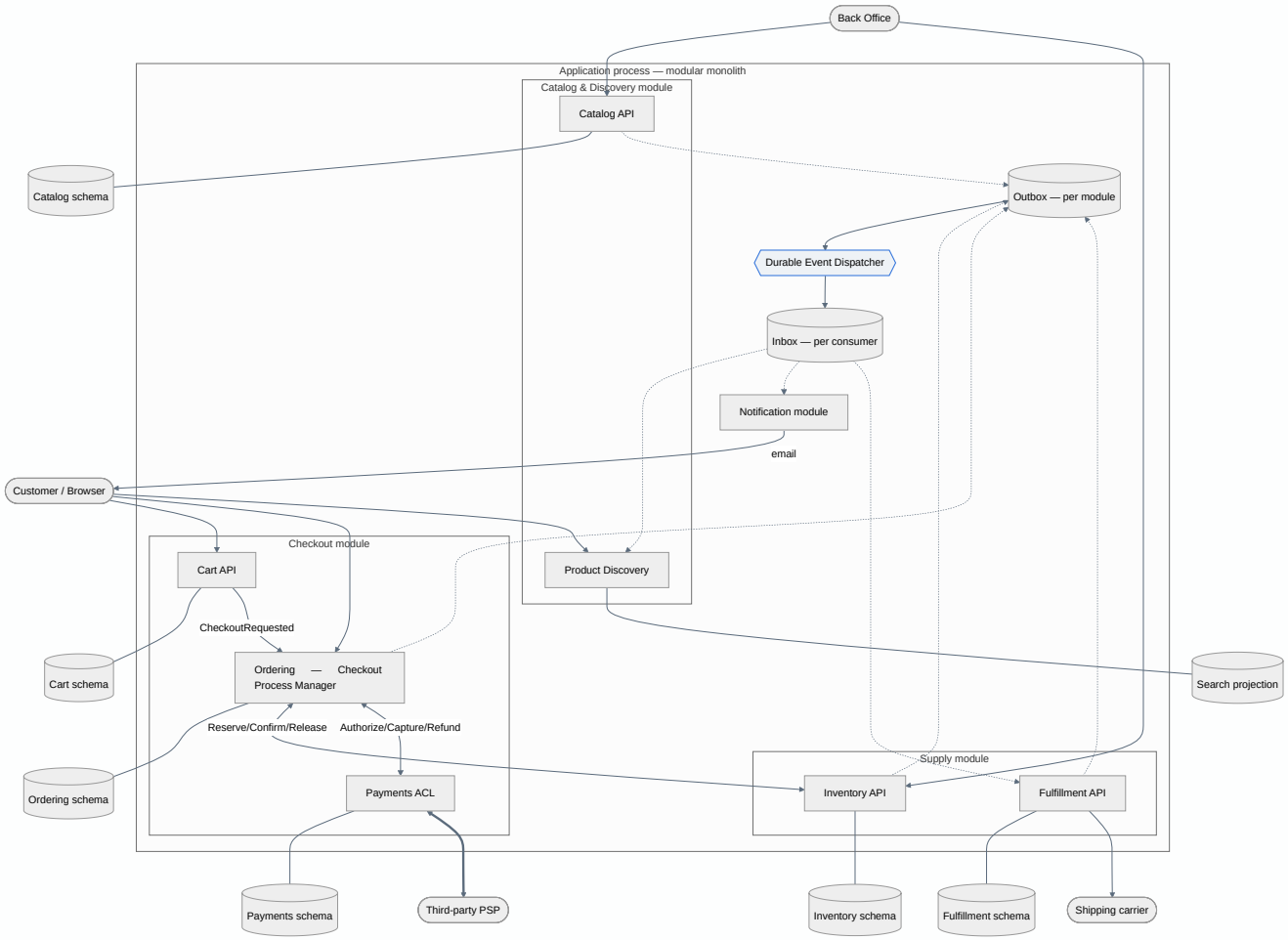



**Table 4 — Team ownership**

| Team | Owned contexts |
|---|---|
| Catalog & Discovery | Catalog, Product Discovery |

| | |
|---|---|
| Checkout | Cart, Ordering, Payments |
| Supply | Inventory, Fulfillment |
| Customer Communication | Notification (or initially shared ownership with Checkout) |

The monolith initially remains a single deployment unit: teams must coordinate the application release, but they do not share aggregates or modify one another's internals. Public contracts and compatibility tests allow modules to evolve with less coordination — they do not eliminate all deployment coordination.

## 6 Impact, Quality, and Effort

Every assumption in the assignment has a dedicated mechanism:

**Table 5 — Requirement coverage**

| Requirement | Where it is addressed |
|---|---|
| Search by name and description | Full-text projection, Figure 5 |
| Add to cart if available | Best-effort check on `inStock`; authoritative reservation at checkout (Figure 2) |
| Back Office: product with image, description, quantity; restocking | `Product` and its related `StockItem.receiveStock()`, Table 2 |
| Checkout with address and payment method, including guest checkout | `Order.Buyer` with a required email and optional `customerId` |
| Payments through a third party | `Payments` with ACL, idempotency, and reconciliation (Figure 4) |
| Out of stock, prevent payment and report it in the UI | The payment service is called only after all reservations succeed; partial success is compensated (Figure 3) |
| Stock managed outside Payments | `Inventory` does not know the PSP; `Payments` does not know stock levels |
| Preparation, shipping, and related notifications | `Fulfillment` manages shipping; `Notification` communicates events to the customer (Figure 5) |

The design protects trust in the ordering process: customers are charged only after stock is reserved and confirmed, and no item can be sold twice.

Scalability comes from read projections and product-level write isolation. Resilience relies on outbox/inbox patterns, idempotency, reconciliation, TTLs, and compensations. Asynchronous projections, notifications, and fulfillment keep the system responsive, while the process manager makes checkout progress observable during slow payment service responses.

Trade-offs are explicit: abandoned checkouts temporarily hold stock, availability may be briefly stale, and compensations add complexity to avoid distributed transactions. Monitoring must cover expired reservations, stuck processes, uncertain payment service outcomes, and outbox backlogs.

Bounded contexts own separate aggregates. A modular monolith supports a small team and explicit contracts; modules should be extracted only when justified by load, compliance, or organizational autonomy.